

Mission 004 Backend Approach

Goal

Make the displayed timeline reflect the updated truth by introducing a real ingestion path into the canonical production source.

Approach

Keep the current read-only response shape stable, but replace any frozen or manually curated source with a production-safe ingestion pipeline. The backend should accept data from the real source, normalize it, persist it in the canonical store, and continue to serve the latest mission only with stable ordering, parallel metadata, and freshness fields.

Backend responsibilities

- Keep ``GET /api/missions/latest`` read-only
- Preserve the current response contract
- Serve the latest mission only
- Keep chronological ordering stable
- Preserve ``sequenceIndex``, ``parallelGroupId``, ``parallelOrder``, and ``parallelSize``
- Preserve freshness metadata (``freshnessState``, ``isStale``, ``lagMs``, and related fields)
- Implement the ingestion path that updates the canonical production source
- Make the ingestion method explicit through environment or runtime configuration
- Keep cache behavior production-safe and predictable

Production source decision

The backend should use a canonical production source, not a frozen repo-backed snapshot, for the final mission. Safest practical path in this repo: poll a configured HTTPS JSON endpoint via ``AGENTS_VIS_DASHBOARD_SOURCE_URL`` and normalize it into the canonical source shape. Webhook is optional, not required. If no external live JSON endpoint is available yet, that gap remains a blocker for Mission 004 completion; the bundled source can keep local and production reads stable until the endpoint is supplied.

Verification targets

- Ingestion writes fresh events into the canonical source
- ``GET /api/missions/latest`` returns the updated production source payload
- Response shape remains stable
- Lag / stale information is present and accurate
- Parallel metadata survives the source swap
- Backend tests cover the ingestion path and contract stability